Infrastructure as Code (IaC) w erze chmur

Definicja IaC

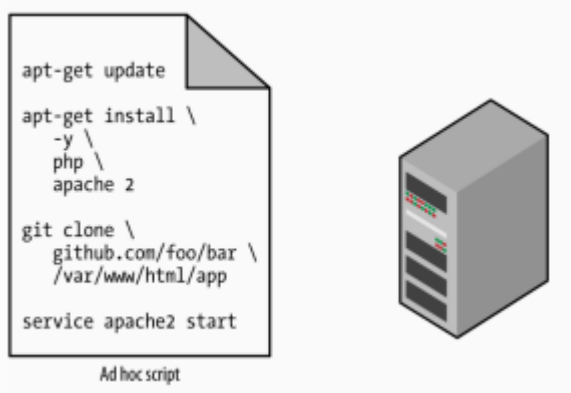
- pozwala na definiowanie, wdrażanie, modyfikacje oraz usuwanie infrastruktury poprzez pisanie i wykonanie kodu
- deklaratywny styl pisania kodu
- dobrze pasuje do zwirtualizowanego hardwaru
- reguły:
  - kontrola wersji - odtwarzanie zmian
  - przewidywalność - zawsze to samo środowisko
  - spójność - wiele instancji tego samego schematu są bardzo podobne do siebie
  - modułowość

Style IaC

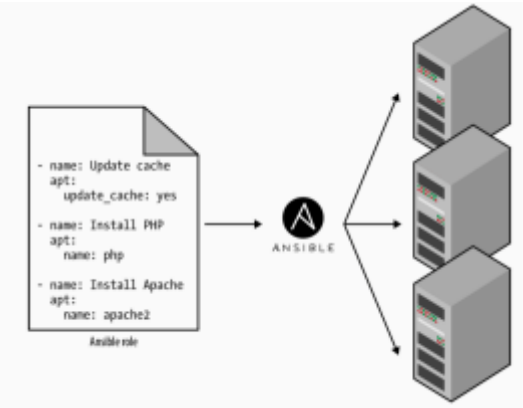
- *what* - podejście deklaratywne, opisujemy stan docelowy
- *how* - podejście imperatywne, opisujemy kroki wykonania
- provisioning:
  - tworzenie i modyfikacja zasobów przez API dostawców
- immutable infrastructure:
  - zamiast ręcznie zmieniać serwer, budujemy nowy obraz lub kontener
  - ogranicza problem *configuration drift*

Kategorie narzędzi IaC

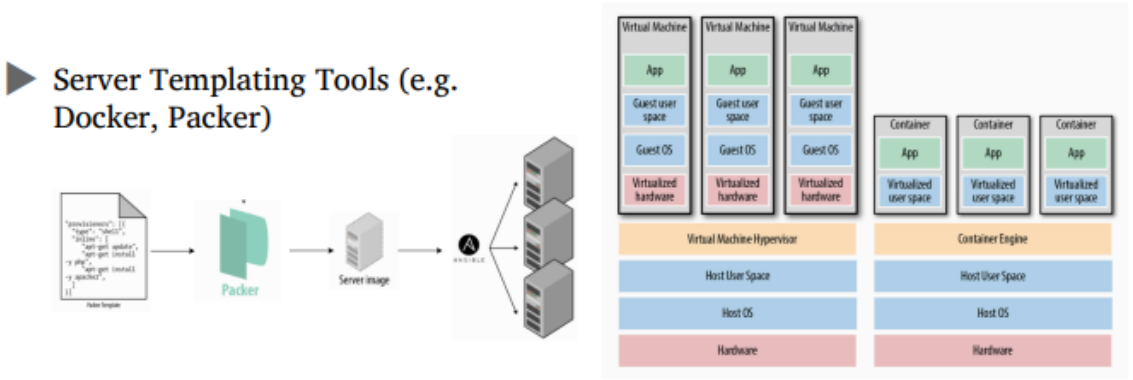
- skrypty ad-hoc:



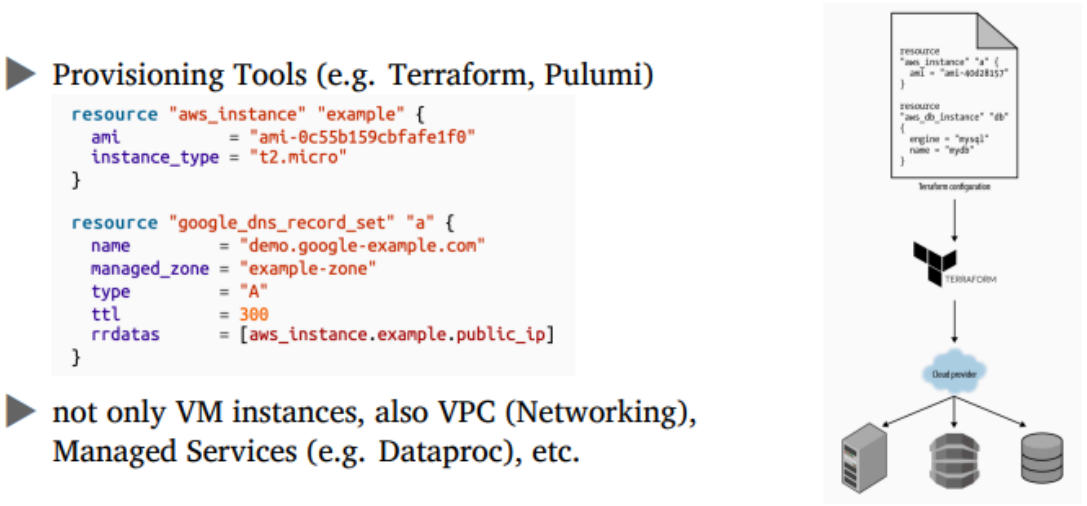
- narzędzia zarządzające konfiguracją:



- narzędzia ze schematami serwerów:



- narzędzia z providerami (*provisioningu*):



Terraform

- narzędzie tworzące architekturę
- przystosowane do chmury
- deklaratywny język programowania HCL

Podstawowe komendy

- `init` - przygotowanie katalogu roboczego na inne komendy
- `validate` - sprawdź czy konfiguracja jest poprawna
- `plan` - pokaż zmiany wymagane przez obecną konfigurację, generuje plan działania i wizualizuje zmiany
- `apply` - stwórz lub zmodyfikuj infrastrukturę (wykonaj zmiany pokazane przez `plan`)
- `destroy` - usuwa **tylko i wyłącznie** obiekty aktualnie zarządzane przez terraform i zapisane w pliku `terraform.tfstate`

Dostawcy (*providers*)

- zawierają typy **zasobów** oraz **danych**, dzięki którym można komunikować zmiany do zdalnych API
- posiadają drobne lokalne narzędzia, np. do generowania losowych liczb do unikalnego nazywania zasobów

```
provider "google" {
  project = var.project_name
  region  = var.region
}
terraform {
  required_providers {
    google = {
      version = "~> 4.8.0"
    }
    random = {
      source  = "hashicorp/random"
      version = "3.1.2"
    }
    kubectl = {
      source  = "gavinbunney/kubectl"
      version = "1.14.0"
    }
  }
}
```

Zasoby (*Resources*)

- każdy zasób ma wejścia (argumenty) i wyjścia (atrybuty)
- do atrybutów (wyjść) zasobów można się odwoływać w innych zasobach
- wyliczone atrybuty są dostępne tylko po utworzeniu zasobu (np. URL albo ID)

Źródła danych

- reprezentują informacje *read-only* pobierane od dostawców

Zmienne

- wejściowe i wyjściowe
- przesyłane jako zmienne źrodowiskowe lub w pliku `.tfvars`
- mogą być lokalne, tylko dla konkretnych modułów
- mogą mieć walidację
- można definiować `locals` do wielokrotnego użycia wyrażeń

Zależności

- niejawne:
  - wynikają z odwołań do atrybutów innych zasobów
- jawne:
  - `depends_on` wymusza kolejność tworzenia lub usuwania

Zarządzanie stanami

- lokalne (domyślnie)
  - plik `terraform.tfstate` w formacie JSON
  - niebezpieczne i podane na błędy
- pamięć współdzielona
  - wymaga zdalnego backendu, np. GCS
  - szyfrowana komunikacja
  - wersjonowanie
  - współpraca z zespołem

Moduły

- każdy zestaw plików konfiguracyjnych Terraform w katalogu jest modułem

- zawsze mamy przynajmniej moduł `root`
- reużywalność kodu
- można go przechowywać w gicie
- stosy
  - kolekcje modułów które są logicznie powiązane ze sobą
  - reprezentują pojedynczą jednostkę wdrożenia infrastruktury, np. środowisko, lub jego część (system plików)
  - ograniczają zakres zmian w zasobach
  - szybsze niż zarządzanie stanami wszystkich zasobów w jednym pliku
  - rozdzielenie odpowiedzialności

Pętle i wyrażenia

- `count`
  - przydatny gdy chcemy utworzyć wiele podobnych zasobów
- `for_each`
  - wygodny gdy każdy zasób identyfikujemy kluczem lub nazwą
- wyrażenia warunkowe
  - przydatne do opcjonalnych modułów i parametrów

Terragrunt

- orkiestrator
- upraszcza reużywalność kodu IaC i definicję środowisk
- każda jednostka może mieć własny stan i własny plik `terragrunt.hcl`